



南开大学
Nankai University

南 开 大 学

计 算 机 学 院
算法导论大作业实验报告

无人机配送路径规划问题

姓名：吕昊远

学号：2410668

专业：计算机科学与技术

指导教师：苏明

2026 年 6 月 16 日

目录

一、 实验背景 1

二、 问题描述 1

三、 算法原理 1

 (一) 暴力算法 1

 (二) Held-Karp 状态压缩动态规划 2

 (三) 动态规划与状态空间剪枝 2

四、 程序设计与实现 3

 (一) 暴力枚举 3

 (二) Held-Karp 动态规划 3

 (三) 状态剪枝优化 4

 (四) 样例数据与测试 5

五、 实验结果分析 5

 (一) 测试环境 5

 (二) 结果总览 6

 (三) 并行扩展性 7

 (四) 算法分析 8

六、 算法与工程实践 10

 (一) 近似求解 10

 (二) 效率对比 11

七、 问题探讨 12

八、 总结与体会 13

一、 实验背景

近年来，随着低空空域逐步开放，“低空经济”成为新的产业风口，无人机配送是其中最贴近日常生活，且正在多地逐步试点落地的场景之一。相比地面上的骑手，配送无人机不受路面交通约束，可以走近似直线的空中航线，在日常生活中的即时配送服务中，展现出了明显的时效优势。

对于未来的无人机配送服务，主流考虑采用分布式的配送站点来实现。一个典型的运行模式是：配送站一次性装载若干份订单，派出一架无人机依次飞往各个客户的配送点，订单完成投递后返回，继续进行下一次配送。

由于即时配送对于送达时间有着极强的约束，每份订单都有承诺的最晚送达时刻，因此调度系统必须在保证不超时的前提下，尽快送达整批订单。这就自然地提出了一个核心的问题：**给定一批客户订单（包含配送点与最晚送达时限），如何为单架无人机安排最优的送餐顺序，使其总完成时间最短。**

本次实验即围绕这一问题展开。从算法的视角，系统地设计、实现并比较多种求解方法，同时进一步探讨其在工程方面的实践意义。通过测试与分析，探究算法在现实场景中的应用，以及算法思想为我们带来的启发。

二、 问题描述

对于这样一个问题，我们可以使用图结构来构建模型。

将起降站记为节点 0， k 位客户（即 k 份订单）记为 $1, \dots, k$ 。符号约定如下： L_i 为客户 i 的最晚送达时刻； t_{ij} 为 $i \rightarrow j$ 的飞行时间，等于两点欧氏距离除以航速； θ 为在每个客户点停留的卸餐服务时间（常数，可取 0）。

一条航线是客户的一个排列 $\pi = (\pi_1, \dots, \pi_k)$ ，对应路径 $0 \rightarrow \pi_1 \rightarrow \dots \rightarrow \pi_k \rightarrow 0$ 。沿途到达第 m 个客户的时刻为飞行与服务时间的累加：

$$A_{\pi_m} = \sum_{l=1}^m t_{\pi_{l-1}\pi_l} + (m-1)\theta, \quad \pi_0 = 0. \quad (1)$$

该航线可行当且仅当每位客户 i 满足 $A_i \leq L_i$ 。优化的目标是最小化总完成时间，即无人机返回站点的时刻：

$$\min_{\pi \text{ 可行}} \left(\sum_{m=1}^k t_{\pi_{m-1}\pi_m} \right) + t_{\pi_k 0} + k\theta. \quad (2)$$

通过上述对问题的建模，我们可以发现，该问题即为**带时间窗的旅行商问题**（Traveling Salesman Problem with Time Windows, TSPTW）。考虑到现实场景，问题中的订单只有“最晚送达”的约束、没有“最早送达”的约束。因此配送途中无需等待，到达时刻等于飞行与服务时间的纯累加，即**总完成时间沿路径可加**。这一可加性是后续算法得以成立的基础。

三、 算法原理

（一）暴力算法

对于该问题，最简单直接的思路就是进行暴力枚举，即枚举全部 $k!$ 个客户访问顺序，对于每个顺序依照式 (1) 递推计算送达时刻，检验全部时限，再按照式 (2) 计算可行顺序的总

完成时间，取其中最小者作为最优解。由于穷举了整个解空间，暴力算法必然能得出真正的最优解，因而可以作为衡量其他方法正确性的标准答案。每个排列的评估为 $O(k)$ ，遍历一共得到 $k!$ 个排列，故其时间复杂度为：

$$O(k! \cdot k). \quad (3)$$

阶乘复杂度增长极快，暴力算法仅在小规模数据范围（约 $k \leq 11$ ）下可行。其价值主要在于提供一个精确的基准，而非实际使用时的性能。

(二) Held-Karp 状态压缩动态规划

观察暴力算法的过程，我们可以发现，其浪费的地方主要在于需要把整条顺序都记录下来。但是实际过程中，从某一刻起，未来如何行进仅仅取决于**还剩哪些客户未送与当前停在何处（及到达时刻）**，与此前已经送达的客户的访问次序无关。因此，凡是已送达客户集合 S 相同、且都停在同一终点 j 的不同顺序，都可以压缩为同一个状态，求解时只需保留其中到达 j 最早的即可。这也正是动态规划算法中的**最优子结构与重叠子问题** [1]。

定义 $dp[S][j]$ 为“已送完集合 S 、当前停在 $j \in S$ 时到达 j 的最早时刻”，其递推关系为：

$$\text{初始状态: } dp[\{j\}][j] = t_{0j} \quad (\text{仅当 } t_{0j} \leq L_j), \quad (4)$$

$$\text{转移方程: } dp[S][j] = \min_{i \in S \setminus \{j\}} (dp[S \setminus \{j\}][i] + \theta + t_{ij}) \quad (\text{保留当且仅当 } \leq L_j), \quad (5)$$

$$\text{最终结果: } \min_j (dp[\{1, \dots, k\}][j] + \theta + t_{j0}). \quad (6)$$

该算法的正确性来自其可加性，在式 (5) 中子集最优与末段组合得到整体最优，且标量最早到达时刻完全足以刻画一个状态的全部后续信息。在复杂度方面，状态数为 2^k 个子集乘以 k 个终点，即 $O(2^k k)$ ；每个状态转移时至多枚举 k 个前驱，于是有：

$$\text{时间复杂度: } O(2^k k^2) \quad , \quad \text{空间复杂度: } O(2^k k). \quad (7)$$

状态压缩 DP 将问题规模从阶乘降到了指数，当 $k = 18$ 时， $18! \approx 6.4 \times 10^{15}$ ，而 $2^{18} \cdot 18 \approx 4.7 \times 10^6$ ，两者相差约 9 个数量级，完全可以满足现实中大部分配送场景的需求。

(三) 动态规划与状态空间剪枝

在上述 DP 算法的基础上，还可以对其进行剪枝优化，使其可以在保持最优解不变的前提下大幅减少实际访问的状态数，从而提升算法的效率。

时间窗状态剪枝. 考察状态 (S, j) 在时刻 $\tau = dp[S][j]$ 。对任一未送达客户 v 来说，其能被送达的最早可能时刻为立即直飞的结果，即 $\tau + \theta + t_{jv}$ （欧氏距离满足三角不等式，绕行只会更晚）。如果连该最早时刻都已经超过 L_v ，则 v 此后一定不可能按时送达， (S, j) 为死状态，可以将其整体剪去。

下界分支限界. 维护当前已知最优完整可行解 I (incumbent)，并为每个状态估算完成剩余行程的下界 LB ；若 $\tau + LB \geq I$ ，则经过该状态不可能得到更优的解，剪去。在这里要注意的是，下界必须为可采纳的（永不高估）。在过程中，从 (S, j) 完成行程需在 j 卸餐、飞

经全部剩余客户 R 、各自卸餐并返航；服务部分 $(|R| + 1)\theta$ 为精确值，飞行部分我们以各点最小入边求和作为下界：

$$LB = (|R| + 1)\theta + \sum_{v \in R} \min \text{In}[v] + \min \text{In}[0], \quad \min \text{In}[v] = \min_{u \neq v} t_{uv}. \quad (8)$$

其可采纳性在于，任何真实的剩余路径都恰好为 R 中各点与站点 0 各自提供了一条入边，而每条入边一定不小于该点最小入边，逐点求和不超过真实飞行代价。由于下界从不高估，被剪状态都已被证明无法优于 I ，可证最优解不会被误剪，故其精确性保持不变。这一“Held-Karp 状态压缩 DP + 时间窗状态剪枝”的精确求解思路与 Dumas 等 [2] 一致。

四、 程序设计与实现

(一) 暴力枚举

暴力枚举算法的实现较为简单,直接枚举解空间进行计算即可。借助 `std::next_permutation` 从字典序最小的排列开始遍历，保证不重不漏；对每个排列做一次线性递推评估。

Algorithm 1 暴力枚举 (BruteForce)

Input: 客户集合 $\{1, \dots, k\}$, 时限 L , 时间表 t , 服务时间 θ

Output: 最优总完成时间 $best$ 与最优排列 $bestPerm$

```

1:  $best \leftarrow +\infty$ ;  $bestPerm \leftarrow \emptyset$ 
2: for all 排列  $\pi$  of  $(1, \dots, k)$  do
3:    $flight \leftarrow 0$ ;  $service \leftarrow 0$ ;  $prev \leftarrow 0$ ;  $feasible \leftarrow \text{true}$ 
4:   for  $m \leftarrow 1$  to  $k$  do
5:      $flight \leftarrow flight + t_{prev, \pi_m}$ ;  $arrival \leftarrow flight + service$ 
6:     if  $arrival > L_{\pi_m}$  then
7:        $feasible \leftarrow \text{false}$ ; break
8:     end if
9:      $prev \leftarrow \pi_m$ ;  $service \leftarrow service + \theta$ 
10:  end for
11:  if  $feasible$  and  $flight + t_{prev, 0} + service < best$  then
12:     $best \leftarrow flight + t_{prev, 0} + service$ ;  $bestPerm \leftarrow \pi$ 
13:  end if
14: end for
15: return  $best$ ,  $bestPerm$ 

```

(二) Held-Karp 动态规划

这部分的实现思路与常规的状态压缩 dp 基本一致，我们采用整型比特掩码表示集合 S (客户 c 占第 $c-1$ 位)。由于加入一个客户必使掩码数值增大，直接按照掩码数值递增的顺序进行遍历天然满足拓扑序；实现上采用前向 push 更新，最优路线由 dp 表回溯重建，无需构建额外的前驱数组。

Algorithm 2 Held-Karp 动态规划 (HeldKarp)

```

1:  $FULL \leftarrow 2^k - 1$ ;  $dp[\cdot][\cdot] \leftarrow +\infty$ 
2: for  $j \leftarrow 1$  to  $k$  do
3:   if  $t_{0j} \leq L_j$  then
4:      $dp[\{j\}][j] \leftarrow t_{0j}$ 
5:   end if
6: end for
7: for  $mask \leftarrow 1$  to  $FULL$  do ▷ 掩码递增即拓扑序
8:   for all  $j \in mask$  with  $dp[mask][j] < +\infty$  do
9:     for all  $nj \notin mask$  do
10:       $arr \leftarrow dp[mask][j] + \theta + t_{j,nj}$ 
11:      if  $arr \leq L_{nj}$  then
12:         $dp[mask \cup \{nj\}][nj] \leftarrow \min(dp[mask \cup \{nj\}][nj], arr)$ 
13:      end if
14:    end for
15:  end for
16: end for
17: return  $\min_j (dp[FULL][j] + \theta + t_{j0})$ 

```

并行实现. 在上述的 push 转移中, 同一集合不同终点会写入同一目的状态, 多线程并发更新存在数据竞争的问题。对其进行并行编程时需要改为 pull 的转移方式, 并按 $|S|$ (即 popcount) 分层。具体实现上, 每个状态主动从“恰少一个客户”的前驱拉取最小值。由于第 $|S|$ 层只读第 $|S| - 1$ 层、只写本层, 同层各状态写入互不相交, 不存在锁的问题, 我们可以直接使用 OpenMP 对该层全部掩码进行多线程并行; 层间的隐式屏障则对应层依赖。pull 转移中先取最小再判时限与 push 转移给出的 dp 值完全相同, 故并行版与串行版最优值及保留状态数均相等, 结果完全一致。

(三) 状态剪枝优化

剪枝优化后的大体框架基本相同, 仅在处理“是否扩展某状态”时加设两处条件判断, 分别为时间窗剪枝和下界限界。其中 incumbent 初值由贪心启发式 (最近邻、最早时限两种取优) 给出, 并可经一轮 2-opt 局部搜索 [3] 收紧 (详见后续第六节讲解)。

Algorithm 3 DP + 剪枝的状态处理 (替换 DP 算法内层)

-
- 1: 预计算 $\text{minIn}[\cdot]$; 由启发式得到 incumbent I
 - 2: 取出可达状态 (S, j) , $\tau \leftarrow dp[S][j]$, $R \leftarrow \{1, \dots, k\} \setminus S$
 - 3: **if** $\exists v \in R: \tau + \theta + t_{jv} > L_v$ **then**
 - 4: 剪去 (时间窗剪枝); 处理下一状态
 - 5: **end if**
 - 6: 由式 (8) 计算 LB
 - 7: **if** $\tau + LB \geq I$ **then**
 - 8: 剪去 (下界限界); 处理下一状态
 - 9: **end if**
 - 10: 否则照 Algorithm 2 内层向 $n_j \notin S$ 扩展
-

(四) 样例数据与测试

为了让我们的实验数据更加全面严谨, 同时由于算法策略较多, 我们采用了批量测试脚本的方式来进行测试。在 AI 的帮助下, 对 `main.cpp` 改写得到了一版使用命令行进行参数设定的测试程序入口 `bench.cpp` 以及相应的测试脚本 `run_bench.py`; 在数据生成方面, 由于现有的公开数据集多样性较差, 对应在多种算法上难以全面体现其差异特征, 我们也在 AI 的帮助下编写了用于生成测试数据的 `gen.cpp`, 随机生成具有不同特征与规模的测试样例以供测试。

此处仅为测试框架的说明, 由于并非算法重点, 这里不作多余的阐述。详见代码仓库链接。

五、实验结果分析**(一) 测试环境**

实验测试在本地开展。为减少性能波动, 相关算法的测试均在本机的 Ubuntu 虚拟机中完成, 结果的统计分析与绘图则在宿主机上进行。环境配置如下:

表 1: 实验测试环境

处理器	VMware 虚拟平台, 分配 4 个 vCPU
内存	4 GB
操作系统	Ubuntu 25.10 (内核 6.17.0-22-generic)
编译器	g++ 15.2.0
编译指令	-std=c++17 -O2 -fopenmp
Python	3.10.11
依赖库	numpy 1.26.4、pandas 2.3.3、matplotlib 3.10.9

(二) 结果总览

我们首先以两组具有代表性的测试样例对比各个算法, 分别为一个小规模实例 ($k = 10$, 聚簇分布、松时窗, 表 2) 和一个较大规模实例 ($k = 16$, 同为聚簇分布、紧时窗, 表 3), 二者的平均延迟对比如图 1 所示。这里我们以 **串行 Held-Karp DP** 的平均延迟作为基准计算加速比。

表 2: 不同算法在小规模实例 ($k = 10$, clustered, loose) 上的测试结果

方法	最优值	访问状态数	平均延迟/ms	加速比
暴力枚举	34.3283	3 628 800	65.74	-
Held-Karp DP	34.3283	5 120	0.320	1.00
并行 DP	34.3283	5 120	0.231	1.38
DP + 剪枝	34.3283	2 630	0.430	0.74

如表 2 所示, 四种方法返回的最优值完全一致 (34.3283), 从结果层面验证了各算法的正确性。性能层面, 各个算法在规模与速度上则具有较大差异。暴力算法需枚举 $10! = 3\,628\,800$ 个排列, 而 Held-Karp DP 仅保留 5 120 个可达状态 (约为前者的 $1/700$), 延迟随之由 65.74 ms 降至 0.32 ms, 速度提升了超过两个数量级, 具有明显的效率优势。

然而, 观察数据可以发现一处反常的现象, 在该实例上, DP + 剪枝虽然把状态数进一步压缩到了 2 630, 但其延迟却略高于纯 DP ($0.430 > 0.320$)。实际上, 其原因主要在于剪枝时的额外开销。通过贪心初值、一轮 2-opt 局部搜索与逐状态的下界计算, 这一系列流程在实例规模小、松时窗场景下, 状态数减少带来的收益反而被该部分的固定开销所抵消了。

为此, 我们还选取了规模更大、时窗更紧的实例 ($k = 16$, 同为聚簇分布、tight) 作为对照, 结果如表 3。对于这样规模的测试样例, 暴力几乎没有可行性 ($16! \approx 2.1 \times 10^{13}$), 故只对三种 DP 方法进行比较。

表 3: 不同算法在较大规模实例 ($k = 16$, clustered, tight) 上的测试结果

方法	最优值	访问状态数	平均延迟/ms	加速比
Held-Karp DP	162.4426	401 576	38.09	1.00
并行 DP	162.4426	401 576	17.98	2.12
DP + 剪枝	162.4426	77	4.95	7.70

当来到大规模的实例, 剪枝带来的收益就变得尤为显著。在该实例中, DP + 剪枝把访问状态数从 401 576 大幅缩减到 **77** (减去的状态数约为 99.98%), 平均延迟随之由 38.09 ms 大幅降至 4.95 ms, 相对纯 DP 提速 7.70 $\times$, 其效率优势进一步凸显。从这里我们也能看出, 剪枝并非在任何场景下都具有优势, 其固定的开销只有在规模足够大、可剪状态足够多 (tight 实例) 时才能体现出该算法的强大之处。

此外, 并行 DP 则在两组实例中都较为稳定, 通过多线程并行带来常数倍的加速比, 虽然在扩展性上具有一定的边界, 但其对于不同类型的实例来说总能带来稳定的正向的加速收益。

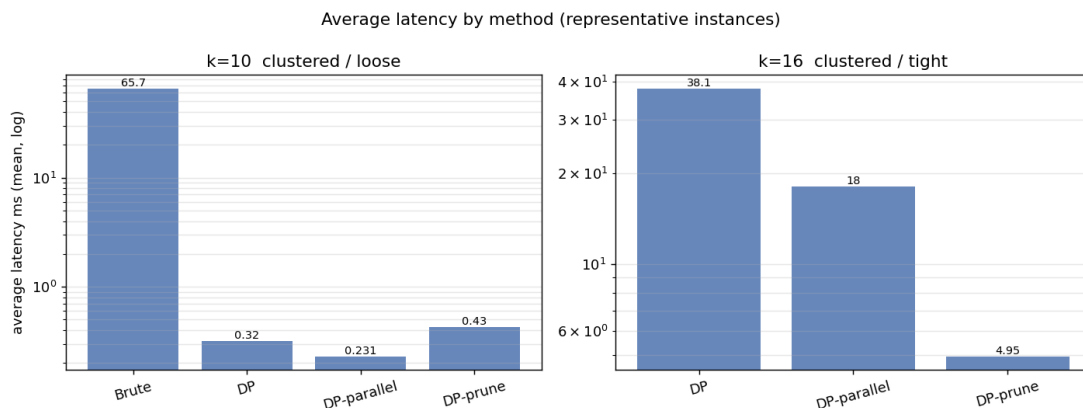


图 1: 各算法在两组实例上的平均延迟

总体来看, DP 算法带来了显著的提升, 将这样一个难以暴力枚举的问题通过算法的思想得到了大幅的优化。同时, 以 DP 为基础的几种算法也与我们的预期较为一致, 在具有不同特征的样例上取得了较为可观的优化效果。接下来我们将对其进行进一步的分析与对比。

(三) 并行扩展性

在并行算法这部分, 与本学期修读的《并行程序设计》这一课程具有一定的重合。我们这里主要实现的是 OpenMP 的多线程编程, 因而其扩展性是我们需要着重考虑的部分。图 2 给出了并行 DP 在固定实例上随线程数变化的扩展性曲线。

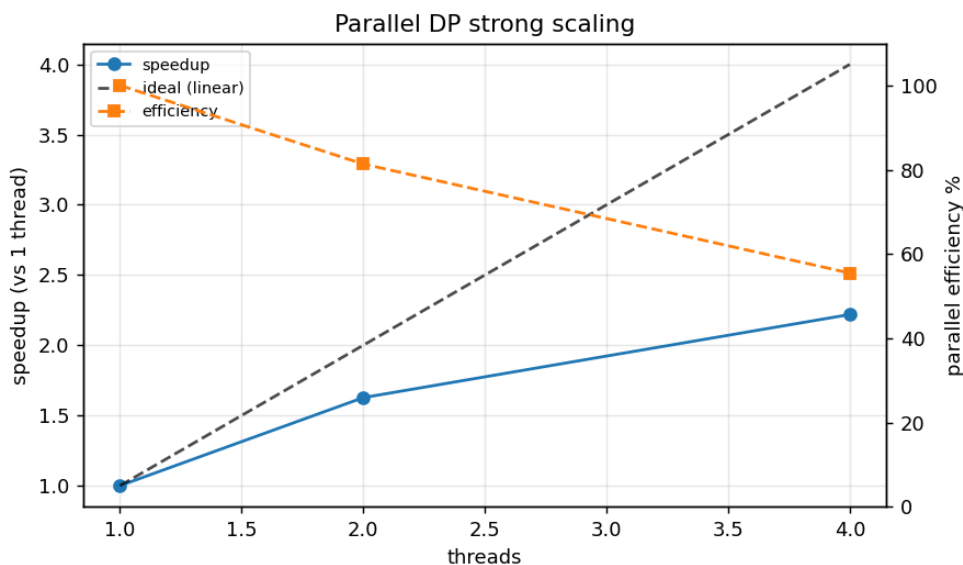


图 2: 并行 DP 的加速比与并行效率随线程数的变化

在实际测量中, 并行算法的加速比为: 2 线程 $1.63\times$ (效率 81.4%), 4 线程 $2.22\times$ (效率 55.5%)。其曲线明显为典型的次线性, 且其并行效率随线程数而单调下降, 这在多线程并行中是一种较为常见的现象。实际上, 从算法的角度来看, 这其实也与分层并行方案自身的逻辑结构相符, 我们可以用 Amdahl 定律 [4] 来解释。

记串行占比为 s , 理想加速比 $S(p) = 1/(s + (1-s)/p)$ 。以实测两点反解, 分别得 $s \approx 0.23$ (由 $p = 2$) 与 $s \approx 0.27$ (由 $p = 4$), 即约有四分之一的工作量本质上是串行的, 对应的渐近加速比上界 $1/s \approx 3.7 \sim 4.4$ 。

经过分析可知, 这部分串行开销主要来自三处: 其一, 第 1 层初始化、末层对 k 个终点取最优、以及路线回溯, 都不在并行区内; 其二, pull 式 DP 严格按 $|S|$ 分层, 第 $|S|$ 层依赖第 $|S| - 1$ 层, 层间存在 K 次隐式屏障同步, 而最低的几层 ($|S| = 2, 3$) 掩码数极少、几乎无可并行的工作, 却同样要承担屏障代价, 从而造成了结构性的负载不均; 其三, dp 表规模为 $2^k \cdot k$ 个 double, pull 过程对其中分散的子集项做随机访问, 属于带宽受限型任务, 线程增多反而会加剧内存带宽的竞争使用。这三者的开销相叠加, 使得在线程数 $T = 4$ 时, 效率直接跌落到了 55% 左右。

在这里我们也能知道, 这一类算法的并行优化往往较为复杂, 需要考虑算法的各处细节从而采用不同的处理策略。虽然并行部分并非我们算法课程的重点, 但是通过简单的分析, 我们也能从中对于算法本身的逻辑得到更加深入的理解, 同时为算法的设计与优化拓宽思路。

(四) 算法分析

在算法的原理中, 访问状态数是该算法一个重要的因素, 为了更加深入地分析算法的效率特征, 这部分我们将进一步考察访问状态数与运行时间随规模 k 的增长 (图 3、图 4), 以及剪枝缩减率随空间分布与时窗松紧的变化 (图 5)。

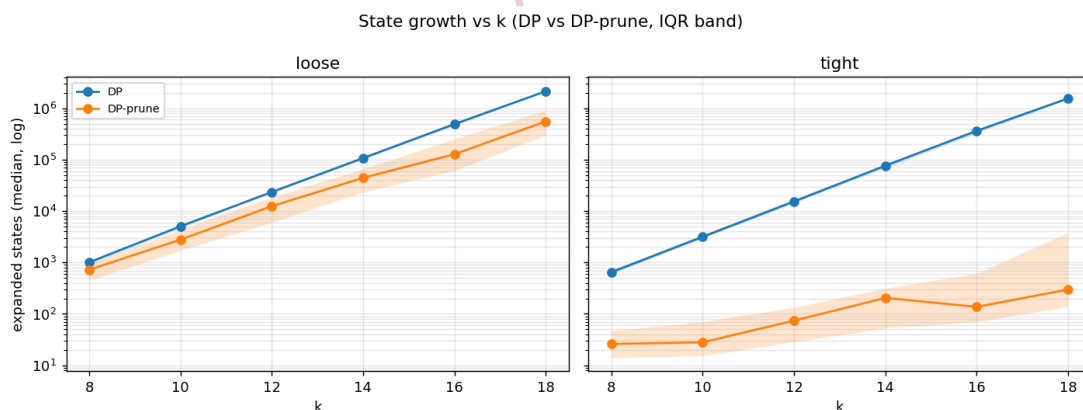
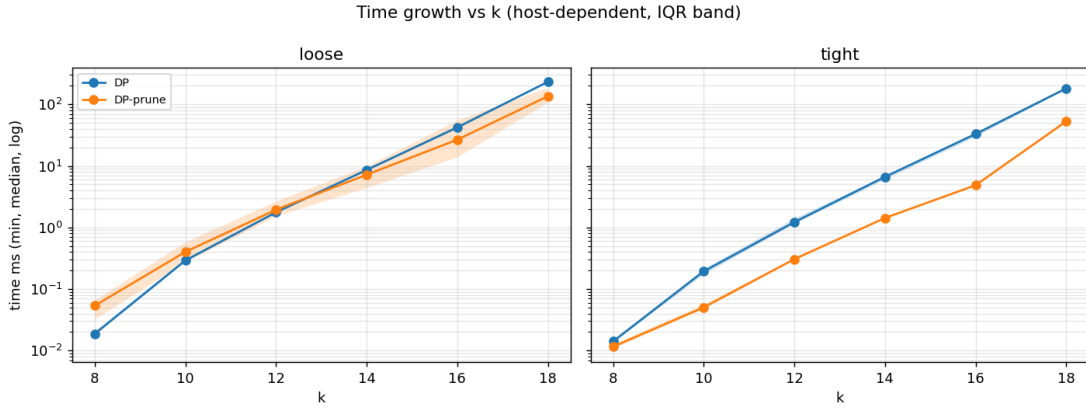


图 3: 访问状态数随规模 k 的增长 (中位数与 IQR 带)

图 4: 运行时间随规模 k 的增长 (中位数与 IQR 带)

观察以上图像, 可以看出其关系特征较为明显。在对数纵轴上, 纯 DP 的状态数随 k 近乎呈直线, 印证了 $\sim 2^k$ 的指数级复杂度。 $k = 18$ 时松、紧两档的可达状态中位数分别约为 2.14×10^6 与 1.55×10^6 , 与 $2^{18} \cdot 18 \approx 4.7 \times 10^6$ 的理论上界同阶 (实测偏低是因为超时不可达的状态未被展开)。对其进行剪枝优化后, 松时窗下状态数降至约 5.54×10^5 (缩减约 74%), 曲线相对纯 DP 下移; 紧时窗下则具有明显的差异, $k = 18$ 的 DP + 剪枝中位数仅 298 个状态, 相对纯 DP 缩减逾 99.98%, 曲线几乎被压平, 明显脱离了 2^k 的增长轨迹。同样地, 运行时间的变化趋势 (图 4) 也与状态数高度一致: 紧时窗 $k = 18$ 下, DP 与 DP + 剪枝的中位耗时分别约为 181.6 ms 与 53.0 ms。

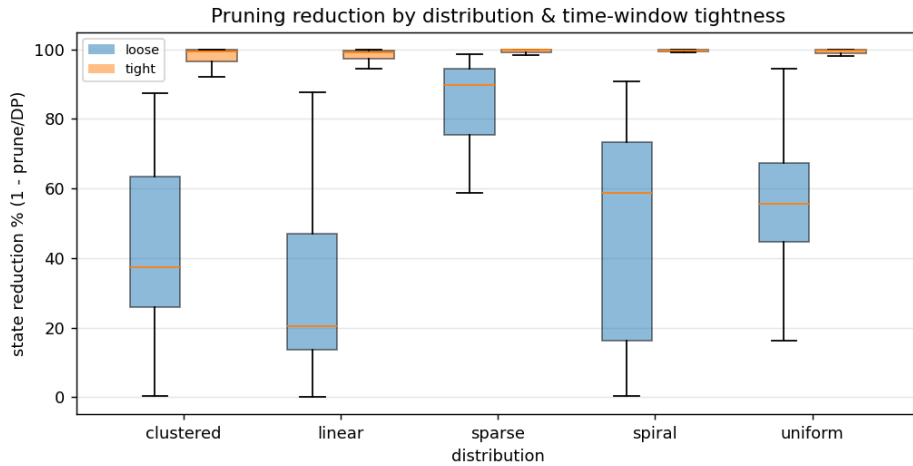
图 5: 不同空间分布与时间窗松紧下的状态缩减率 ($1 - \text{剪枝}/\text{DP}$)

图 5 则清晰地展示了时窗松紧带来的剪枝效果的差异。在紧时窗下, 所有五种分布的缩减率中位数都稳定在 99% 以上 (clustered 99.4%、linear 99.2%、sparse 99.9%、spiral 99.8%、uniform 99.6%), 且分布极为集中。这正是时间窗状态剪枝在起作用, 时间窗口越紧, 就会有越多的状态因为“立即直飞也会超时”而被整体剪去, 从而带来状态数的大幅消减。

而对于松时窗, 时间窗剪枝的效果则相对有限。缩减大部分依赖下界限界, 缩减率随分布大幅波动, 且方差也明显增大。sparse 最高 (中位 89.7%), uniform、spiral 居中 (55.6%、58.8%), 而 linear、clustered 最低 (20.5%、37.4%)。其原因主要在于式 (8) 的最小入边下

界对几何分布较为敏感。当客户点彼此分散 (sparse)，各点最小入边较大、下界较紧，剪枝效果较好；当客户点分布密集或近乎于共线 (clustered、linear)，点间距离彼此接近，最小入边偏小、下界偏松，区分度便随之下降。更紧的可采纳下界（如基于状态空间松弛的方法 [5]）可进一步压缩状态空间，但相应的求界开销也更高，需在界的紧度与求界成本之间加以权衡。

总体而言，实测的增长阶数与 $O(2^k)$ 的理论复杂度、以及剪枝随时窗紧度递增的效果，均与预期高度吻合，通过实测验证了其正确性与合理性。在分析过程中，也体现出这一算法在不同场景下的差异及其优化的边界。

六、 算法与工程实践

经过调研我们发现，在当今大部分的工程系统中常常采用一种“近似”的理念，其类似于搜索问题中的“召回率”这一概念。在许多场景中，往往不需要精确的最优解，只要能控制在略高于最优成本的一定范围内，快速地为大规模实例给出一个较优的解，我们便可以认定该算法具备工程价值。换言之，我们可以为解的质量预留一部分可控的容错空间，以换取整体计算速度的提升，从而使其更加贴合现实工程。针对于此，我们的算法也可以参照这一思路，进行一些工程场景下的扩展。

(一) 近似求解

观察前述精确求解的算法，可以发现在剪枝部分，其中的下界分支限界剪枝具备天然的近似性质。其剪枝判据“若 $\tau + LB \geq I$ 则剪枝”天然提供了一个可以容错的空间。在此基础上，我们引入容错系数 $\varepsilon \geq 0$ ，将判据改为

$$(1 + \varepsilon)(\tau + LB) \geq I \implies \text{剪去}, \quad (9)$$

也就是将剪枝阈值收紧为 $I/(1 + \varepsilon)$ ，从而让算法可以更早、更彻底地剪枝。体现在代码上，仅需一处乘法系数的修改：

```
1 // 精确算法
2 if (tau + LB >= incumbent) { prune(); continue; }
3 // 近似算法
4 if ((1.0 + eps) * (tau + LB) >= incumbent) { prune(); continue; }
```

对于近似的策略，可以证明式 (9) 返回的解 $\leq (1 + \varepsilon) \cdot \text{OPT}$ ，且 $\varepsilon = 0$ 时退化为精确算法。考察最优解所经路径上的状态，其下界必然满足 $\tau + LB \leq \text{OPT}$ 。若其中无一被剪，则 DP 照常求得 OPT；若某状态被剪，则由式 (9) 有 $I \leq (1 + \varepsilon)(\tau + LB) \leq (1 + \varepsilon)\text{OPT}$ ，而返回解不超过 I 。两种情形所得的解均 $\leq (1 + \varepsilon)\text{OPT}$ 。也就是说，从算法原理的角度上来看，其带来的误差一定不会超过我们所设定的阈值范围。

实际上，上述的放松主要取决于 incumbent I ， I 越紧，阈值越早生效、剪枝越强。我们的分层 DP 中 incumbent 为启发式一次性给定，故其质量直接决定近似的提速幅度。为此，在最近邻/最早时限初值上再施加一轮 2-opt 局部搜索来进一步收紧 I ，就可以让近似的剪枝效果在各类实例上稳定发挥其作用。

(二) 效率对比

对近似算法进行同样的测试，得到如下结果。

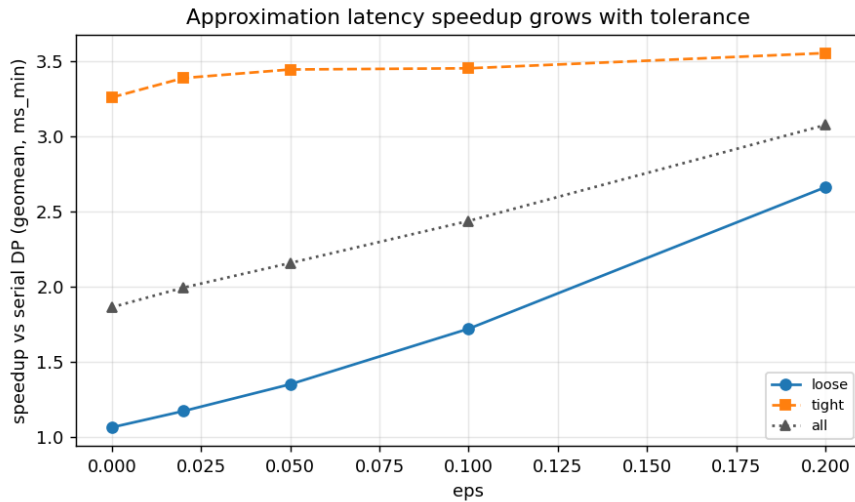


图 6: 近似相对串行 DP 的延迟加速比随容错系数 ϵ 变化

由图 6 可见，随容错系数 ϵ 增大，近似相对串行 DP 的几何平均延迟加速比稳步上升，整体由 $\epsilon = 0$ 的 $1.87\times$ 提升至 $\epsilon = 0.20$ 的 $3.08\times$ （其中 $\epsilon = 0$ 已是带 2-opt incumbent 的精确剪枝，故本就快于纯 DP）。而其松紧时窗的差异也尤为明显，在松时窗实例上，提速主要由 ϵ 实现，加速比从 $\epsilon = 0$ 的 $1.07\times$ 一路攀升到 $\epsilon = 0.20$ 的 $2.66\times$ ；而在紧时窗实例上，加速比在 $\epsilon = 0$ 时即已达 $3.26\times$ ，此后呈现出饱和的趋势（ $\epsilon = 0.20$ 时也仅 $3.56\times$ ）。其原因在于，紧时窗下的提速绝大部分已由精确的时间窗状态剪枝来实现，近似算法的边际收益有限；而松时窗下精确剪枝乏力，恰是近似算法得以发挥的场景，其延迟降低则较为显著。可见 ϵ 近似带来的提速，主要体现在精确剪枝最为吃力的松时窗一端。

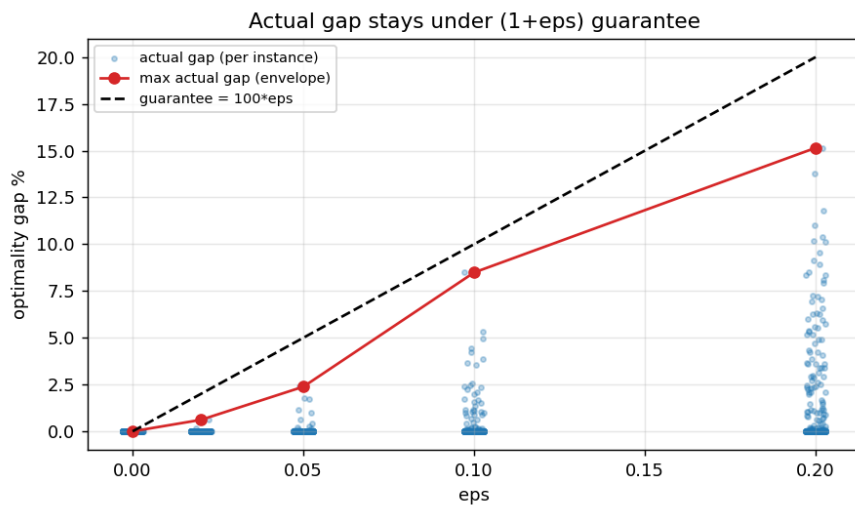


图 7: 实际最优性 gap 与 $(1 + \epsilon)$ 保证的对比

在 $300 \times 5 = 1500$ 个近似样本中，实际 gap 始终位于 $(1 + \epsilon)$ 保证线（ $\text{gap} = 100\epsilon\%$ ）

之下，如图 7 所示。脚本标出了 1 例疑似越界 ($\varepsilon = 0$ 处 $\text{ratio} = 1.000002$) 该值系浮点数舍入所致，而非实际的越界。

在算法上我们确认了近似的保证，而测试中 gap 其实往往远小于其边界。各容差下实际 gap 的最大包络分别为： $\varepsilon = 0.02$ 时 0.61% (保证 2%)、0.05 时 2.38% (保证 5%)、0.10 时 8.49% (保证 10%)、0.20 时 15.15% (保证 20%)，全程低于保证线。更值得强调的是，中位 gap 在所有容差下均为 0，即便取 $\varepsilon = 0.20$ ，仍有 60.3% 的实例返回了精确最优解 ($\varepsilon = 0.10$ 时为 80.7%)。这一现象的根源在于，算法剪枝的依据是 DP 解与 2-opt 收紧后的 incumbent 二者中较优的作为标准，在这批实例上启发式往往已经能命中最优，因而即使收紧后的闸门把通往最优的状态剪去，返回的 incumbent 仍能等于 OPT，其质量不会受损。

对于同一套分支限界框架，仅依靠一个容错参数 ε ，即可在精确最优与快速近似之间简单地进行切换。当 $\varepsilon = 0$ 时是可验证的精确解， $\varepsilon > 0$ 时以可证明的有界次优解换取数量级的加速。这种弹性的取舍，正是算法从理论走向工程实践时最重要的性质。

七、 问题探讨

在本次实验的过程中，也出现了一些值得详细探讨的问题。

1. 问题场景建模时为何选择硬时限？

在本问题的建模过程中，我们选用了硬时间窗（超时即不可行）作为限制，而非将迟到时间计为罚分的软时间窗，这一建模选择直接决定了问题的可分析性与算法策略的选取。考虑实际情况的角度，这两种建模思路都有可取之处，但是也正如课上所讲，即便是微小的问题描述差异，也可能使对应的解题算法产生极大的变化。

在硬时限且无最早约束下，途中无需等待，到达时刻沿路径可加，因而只凭单个标量的最早到达时刻即可完整刻画一个 DP 状态，使 Held-Karp 的递推式 (5) 成立。而如果改为软时限，代价 $\lambda \sum \max(0, A_i - \bar{d}_i)$ 依赖到达时刻本身，“最早到达”不再等价于“代价最小”，一个到达更晚但后续更省的状态可能整体更优，标量状态随之失效。此时需以 (到达时刻, 累计代价) 的二维 Pareto 标号配合支配关系来维护非劣解集，复杂度与实现都显著上升。

其实在日常生活中，准时送达的要求也更为常见，虽然软时限的限制也有一定的道理，但其实际解题过程过于复杂，实现与分析都需进行许多额外的工作。为了简化我们的问题模型，使其可以运用我们课上学到的相关知识来进行设计与分析，加之一些现实因素的考量，我们最终选取了硬时限作为限制进行了问题的建模。

2. 并行扩展性与数据松紧特征的关联

在对数据进行全面分析时可以观察到一个反常的现象，并行 DP 相对串行 DP 的加速比，在松时窗为 1.67 $\times$ ，在紧时窗反而回落到 1.30 $\times$ 。按常理来说，并行加速比主要由线程级并行度决定，与时间窗松紧这类测试实例的属性无关，而结果上却出现了松快紧慢的特征。

深入对比探讨可知，其根源在于串行并行两版算法对于不可达状态的处理方式不同。串行 push 版遇到 $dp[S][j] = \infty$ 的不可达状态会直接跳过不予扩展，故其实际工作量正比于可达状态数；而并行 pull 版为消除写竞争，改为对每一层的全部掩码与全部终点逐一取 min，无论该状态是否可达，都要遍历其前驱，因而其工作量也正比于全部状态数（约 $2^k k$ ）。

实测得到的数据也支持这一解释。串行 DP 自身在 $k = 18$ 紧时窗下的可达状态中位数约 1.55×10^6 ，已低于松时窗的 2.14×10^6 （见图 3），这说明紧时窗的确有更多状态因超时而不可达。于是在紧时窗下，串行版本只需较少的工作，运行速度较快；而并行版仍需遍历所有掩码，为大量不可达状态做了许多无用功，相对一个本已精简的基线，其优势自然有所抵消；而对于松时窗，绝大多数状态均是可达的，两种算法处理的总状态量相近，线程级并行的收益才得以充分体现，这也解释了表 2 中并行版在松实例上以 0.231 ms 明显优于串行 0.320 ms 的结果。

这一现象也揭示了并行优化的边界。当串行版本自身带有足够强大的实例相关的裁剪时，如果并行改写为了规避数据竞争而丢弃了这部分，就很有可能导致在裁剪更为有效的场景（紧时窗）效果变差。

从中也能看出，在 DP 这类问题中，遍历的状态数往往是最为核心的部分，我们的剪枝优化与近似求解的思路也完全围绕于此展开，即便是在并行算法这部分较为常规的优化手段中，状态数的相关分析也能解释许多现象。这其实也为我们指明了方向，在后续的延伸中，也可以围绕状态数来进行各种不同的优化；即便是其他方向的优化思路，状态数也是我们必须深入考虑的关键因素之一。

八、 总结与体会

本次实验以无人机即时配送为背景，将其抽象为带时间窗的旅行商问题，从暴力枚举到状态压缩 DP，再到剪枝优化与近似求解，全面而系统地实现并分析了多种方法。暴力枚举提供精确的基准但因阶乘复杂度不具现实意义；Held-Karp 动态规划凭借最优子结构把阶乘级的穷举遍历压缩到了指数级的状态数；时间窗剪枝与下界分支限界在保持最优的前提下又大幅减少了状态空间；以及容错系数的引入，使其得以扩展为近似求解，为算法赋予了工程意义。

实验数据也支撑了上述主线的每一环。正确性方面，三种精确方法在全部 300 个实例上的 $\max |\text{ratio} - 1|$ 均为 2×10^{-6} ，近似算法在 1500 个样本上也取得了零违反的结果。计算速度方面，纯 DP 呈现清晰的 $\sim 2^k$ 指数增长，剪枝的效果虽然依赖时窗紧度，但其总能带来正向的优化收益，紧时窗下 $k = 18$ 时更是缩减超过 99.98%，曲线近乎压平；并行算法分层 pull 式 DP 也在 4 线程下取得了 $2.22\times$ 加速比，也完全可以由 Amdahl 定律解释；近似算法中，实际 gap 远小于 $(1 + \varepsilon)$ 保证，而探索状态随 ε 大幅消减，印证了这一思路的高效性。

总体而言，本次实验对于涉及各个算法进行了深入的探究，结合课上的知识，进行了系统性的算法设计与分析实验。在过程中，最大的体会在于算法的可解释性与可分析性的探讨。我们的问题之所以能层层递进，每一步都有严谨的理论支撑，其关键在于硬时限带来的可加性，使标量状态足以完整地刻画子问题；也正因如此，剪枝的可采纳性、并行的等价性、近似的有界保证才能都被严格地论证，而非依靠经验直觉来设计。而拥有了扎实的理论基础后，工程化的延伸也自然有了方向，本质上只是把精确证明中的不等式引入了一个因子，即可清晰地转化为工程上的容错与取舍。这也正是算法思想带来的最深刻的启发：先把问题的结构分析清楚，优化与工程延伸往往水到渠成。在解决问题的过程中，时刻保持严谨的态度，既能让我们的思路更具说服力，也能为后续的各种延伸夯实基础。

参考文献

- [1] M. Held, R. M. Karp. A Dynamic Programming Approach to Sequencing Problems. *Journal of the SIAM*, 10(1):196–210, 1962.
- [2] Y. Dumas, J. Desrosiers, E. Gelinas, M. M. Solomon. An Optimal Algorithm for the Traveling Salesman Problem with Time Windows. *Operations Research*, 43(2):367–371, 1995.
- [3] G. A. Croes. A Method for Solving Traveling-Salesman Problems. *Operations Research*, 6(6):791–812, 1958.
- [4] G. M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, vol. 30, pages 483–485, 1967.
- [5] R. Baldacci, A. Mingozzi, R. Roberti. New State-Space Relaxations for Solving the Traveling Salesman Problem with Time Windows. *INFORMS Journal on Computing*, 24(3):356–371, 2012.
- [6] G. Lera-Romero, J. J. Miranda Bront, F. J. Soullignac. Dynamic Programming for the Time-Dependent Traveling Salesman Problem with Time Windows. *INFORMS Journal on Computing*, 34(6):3292–3308, 2022.

附录：代码仓库链接

完整项目见代码仓库链接：<https://github.com/Iriasoul/Algorithms/tree/main>
包含源码与原始数据，以及数据可视化等部分。

附录：AI 使用说明

在本项目中，主要通过 AI 大模型协助完成了测试与数据可视化等工作。instance.h、gen.cpp、gen_sweep.py 与 bench.cpp、run_bench.py 的测试框架有部分使用了 AI 协助完成。以及 analyze.py 的数据可视化，由于对于 Python 不甚熟练，matplotlib 绘图中相对较为复杂的图像也来自于 AI 编码。

除此之外，一些相关算法的论文也由 AI 辅助收集与翻译讲解。其余其他方面 AI 参与的部分也已经在报告中使用下划线标出。

NOTES